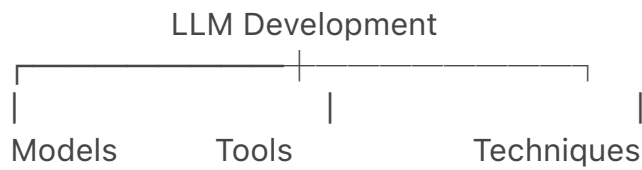


Build your first LLM product

There are three Dimensions of LLMs



Closed-Source Models

These models are hosted by companies.

- Cannot modify weights
- Usually accessed through APIs
- Managed by the provider

Examples:

Company	Model Family
OpenAI	GPT-4.1, GPT-4o, GPT-5, GPT-OSS*
Anthropic	Claude
Google	Gemini
xAI	Grok
Perplexity	Sonar Models

Open-Source Models

- Weights are publicly available.
- Can run locally or on your own server.

Organization	Model
Meta	Llama
Mistral AI	Mistral
Alibaba	Qwen
Google	Gemma
Microsoft	Phi
DeepSeek AI	DeepSeek

Model Selection:

Requirement	Best Choice
Highest reasoning	GPT / Claude
Cheapest	DeepSeek
Local deployment	Llama / Qwen

Small devices	Phi / Gemma
Vision tasks	GPT-4o / Gemini

2:58

1. Week 1 - Build Your First LLM Product: Exploring Top Models

22. Day 3 - Testing ChatGPT-5 and Frontier LLMs Through the Web UI

challenging for LLMs:

1. fresh info where training is not performed yet
2. precision in long derivation maths
3. personal matters
4. extremely long ambiguous emotional/reasoning chain

4:29

1. Week 1 - Build Your First LLM Product: Exploring Top Models

21. Day 3 - Frontier Models: GPT, Claude, Gemini & Their Strengths and Pitfalls

Web search feature is added code (feature) additionally not a part of LLM.

What LLMs Struggle With

- Hallucinations** are probably the most talked-about weakness. An LLM will confidently generate information that sounds completely plausible but is actually false. Tricky part is that the model itself has no real way to distinguish between an answer it's "confident" about and one that's actually "correct". It's just predicting likely-sounding text, not checking facts against reality. This tends to get worse in a few predictable situations: when the model simply doesn't have the relevant knowledge, when your prompt is vague or underspecified, or when the reasoning chain required to answer is long and has more room for the model to drift off track.
- Mathematical computation.** LLMs are fundamentally pattern-predictors, not calculators. They're actually much better at reasoning through a problem conceptually than at doing exact arithmetic. Anything involving large numbers or multiple sequential calculation steps is where things tend to fall apart, since small errors compound. The practical fix is don't make the model do math in its head instead hook it up to an actual calculator or computational tool and let it call that instead.
- Long context**As the amount of text in a conversation or document

grows, model performance quietly degrades and it may start ignoring instructions you gave early on, or lose track of important details buried somewhere in the middle of a long exchange. This is usually addressed through RAG (pulling in only the relevant chunks rather than dumping everything into context) or summarization (compressing older parts of the conversation) commonly.

4. **Real-time knowledge** is a limitation baked into how these models are built. A base model's knowledge is essentially frozen at whatever point its training data was collected. It has no awareness of anything that happened after that cutoff unless it's explicitly connected to external tools like web search. The solution is straightforward: pair the model with web search and RAG so it can pull in current information rather than relying solely on what it learned during training.
5. **Logical consistency**. The same model might give you contradictory answers to related questions, change its stated opinion between prompts, or simply produce outputs that don't line up with each other across a conversation. Temperature (a setting that controls how much randomness goes into the model's word choices) plays a real role here; higher temperature generally means more variation and potentially more inconsistency.

Frontier Models

A frontier model refers to the most capable, state-of-the-art foundation models currently available. The ones built using massive amounts of compute and datasets, pushing the boundary of what's possible. GPT, Claude, Gemini, and Grok are the examples people usually point to. Common characteristics are: these models are general-purpose rather than narrowly specialized, they're multimodal (meaning they can handle more than just text like images, sometimes audio or video too), they support long context windows, they can call external tools, and they tend to show strong reasoning and code generation abilities.

Foundation Model

A foundation model is a large model that's been pretrained on massive data and then serves as the base layer for a wide range of downstream tasks rather than being built for one narrow purpose, it's general enough to be adapted afterward. Example GPT, Llama, Gemini, Claude, and Qwen. The way we adapt a foundation model to a specific task is either through fine-tuning (further training it on task-specific data) or simply through prompting (guiding its behavior with careful instructions, without changing the underlying model at all).

3:41

1. Week 1 - Build Your First LLM Product: Exploring Top Models



26. Day 4 - Understanding Transformers: The Architecture Behind GPT and LLMs

GPT: generative, pre-trained (tons of data from internet), transformer

Transformer (1970s)

"Attention is all you need" by google scientist suggesting data model called neural network, predicting outputs based on parameters & setting them. like lots of stats programs connected together, lots of neurons bind together like stacked capable of computing like humans cannot.

- 2017 - new way to organise - in a architecture good in sequential i/o, looking back and finding what is the important part of input? which should be given attention most - THE ATTENTION LAYER allowing huge scaling
- 2018 - GPT-1, 2019 GPT-2, 2020 GPT-3, 2022 GPT-3.5(chat mode with user), GPT-4 (2023)

there is nothing fundamental about transformer - it was a clever approach which is efficient. there are other ways to approach LLM like before transformer, it was LSTM (long short term memory)

GPT

GPT stands for Generative Pre-trained Transformer, and each of those three words actually describes a distinct part of what the model is.

Generative means the model produces new content rather than just retrieving or classifying existing information including text, code, summaries, SQL queries, and explanations, all generated fresh in response to a prompt.

Pre-trained refers to the fact that the model learns general language patterns from massive datasets before it ever interacts with a real user. During this phase, it picks up grammar, reasoning patterns, factual knowledge, programming syntax, and an understanding of how concepts relate to one another all before any fine-tuning or deployment happens.

Transformer is the underlying architecture that makes all of this possible, introduced back in 2017 in the paper "Attention Is All You Need" created by Google researchers. It replaced RNNs and LSTMs (the older architectures used for sequential data) for most natural language processing tasks, largely because of its attention mechanism, which lets the model weigh the relevance of different words to each other regardless of their distance in the text.

8:13

1. Week 1 - Build Your First LLM Product: Exploring Top Models



27. Day 4 - From LSTMs to Transformers: Attention, Emergent Intelligence & Agentic AI

Agentic AI:

autonomous, master of its own destiny. two ways:

1. LLM responsible for whole workflow including calling other LLMs/tools
2. LLM looping and calling itself with accessing different tools to target particular task.

Like Claude workflow.

0:30

1. Week 1 - Build Your First LLM Product: Exploring Top Models



27. Day 4 - From LSTMs to Transformers: Attention, Emergent Intelligence & Agentic AI

LSTM - sequential, complex and time-taking.

transformer - Attention is all you need, helped parallelise the process. we know the process, but how exactly is it so good under the neural network & very accurate?

0:20

1. Week 1 - Build Your First LLM Product: Exploring Top Models



28. Day 4 - Parameters: From Millions to Trillions in GPT, LLaMA & DeepSeek

traditionally 20-200 parameters/factors. Now 117 million params in GPT-1. Billions, trillions of parameters in new models. Most of the fresh frontier models lost the count..

More params means more intelligence and more training data to absorb, hence computation cost called as **training time scaling**.

Inference time scaling (when you are running the model not training) - smartly prompting using tricks/contexts/forced thinking.

0:38

1. Week 1 - Build Your First LLM Product: Exploring Top Models



29. Day 4 - What Are Tokens? From Characters to GPT's Tokenizer

Tokens

1. Earlier NN were trained at character level (predicting next char in sequence)
2. Then NN trained to predict next word but lead to enormous vocab & rare words omission
3. The the Breakthrough - tokens. Chunk of characters where models recognise fragments of words having some meaning associated with them. moderation between different sequential chunks.

Tokens are passed as as very first inputs to LLMs. Different from vectors which come from deeper in NN. But for both, very first input is still a token ID.

text to Tokens using tokeniser:

some keynotes: it chunks together characters in a sentence which are meaningful together and this grouping creates one token. So a sentence with 50 character can give around 9 tokens approx. (examples of tokeniser used in video)

Also, the number was splitted always in three digits and chunk of three digit created a token.

So 1000 tokens for 750 words (a quarter) is a usual approximation of tokenisation. for maths, code, tokens spike as they need more context.

0:15

1. Week 1 - Build Your First LLM Product: Exploring Top Models



32. Day 4 - Context Windows, API Costs, and Token Limits in LLMs

Context window: maximum context it can look in history, particularly important for multishot prompting where the prompt includes examples.

Costs of API depends on number of i/o tokens per API call. hence they are not per usage charge.

If some info is cached and resent to models, it will charge less due to caching of information

0:52

1. Week 1 - Build Your First LLM Product: Exploring Top Models



31. Day 4 - Tokenizing with tiktoken and Understanding the Illusion of Memory

Code to display token mappings:

```
1 | import tiktoken
2 | encoding=tiktoken.encoding_for_model("gpt-4.1-mini") //getting encoder
  | from tokeniser of given model
3 | tokens=encoding.encode("hi there!")//using the encoder
4 | //it gives us a list of token_ids
5 |
6 | for token_id in tokens:
7 |     token_text=encoding.decode([token_id])
8 |     //token_text represent words/text mapped by token_id
```

hence, tiktoken encoding_for_model gives us encoding and decoding function for a sentence.

4:38

1. Week 1 - Build Your First LLM Product: Exploring Top Models



36. Day 5 - Building a Brochure Generator with GPT-4 and Streaming Results

Stream.

```
stream=openai.chat.completions.create(... stream=true)
for chunk in stream:
```

stream object instead of response format. gets you the response in small, fast parts. It acts like a leaking faucet, giving you water drop-by-drop rather than waiting to fill a whole bucket. You set the parameter `stream=True` in your API request. This stops users from waiting.

4:17

1. Week 1 - Build Your First LLM Product: Exploring Top Models



36. Day 5 - Building a Brochure Generator with GPT-4 and Streaming Results

```
fetch links -> call chat.completions with user+system prompt telling it to
search for relevant links only + json formatting restrictions -> call
chat.completions to create brochure with user+system prompt and md formatting -
> using streaming
```

We can create a single endpoint by reverse calling from the latest function till the earliest function we created.